

Reviewer Pack — Inverse- β decay rate for tropical autoconvolution doubling defect at $n=8$

Entry f26f8d71f2e7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 09:54:17 UTC

Inverse- β decay rate for tropical autoconvolution doubling defect at $n=8$

Entry ID: f26f8d71f2e7 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 09:54:17 UTC

1. Conjecture

Field A (mathematical branch): Maslov dequantization **Field B** (complexity object): Tropical Fourier analysis (Δ doubling-defect of MFC under min-plus self-convolution)

Statement:

Let $f: \mathbb{Z}_8 \rightarrow \mathbb{R}$ be a tropical polynomial and $g = f \otimes_{\min} f$ its min-plus self-convolution on $\mathbb{Z}_8$. Let TFT_β denote the Maslov-dequantized Fourier transform at inverse temperature β , defined by $TFT_\beta(h)[k] = -(1/\beta) \cdot \log \sum_{j=0}^7 \exp(-\beta \cdot h[j]) \cdot \omega^{\{jk\}}$ with $\omega = \exp(2\pi i/8)$, and $MFC_\beta(h) = \min_{k \neq 0} |TFT_\beta(h)[k]|$. Define $\Delta(f, \beta) := |MFC_\beta(g) - 2 \cdot MFC_\beta(f)|$. We conjecture that for every f with entries in $[0, 1]^8$ and every $\beta \in \{5, 10, 20, 50, 100, 200\}$, $\Delta(f, \beta) \leq 10 \cdot \log(8)/\beta$; a single (f, β) pair with $\Delta(f, \beta) > 10 \cdot \log(8)/\beta$ falsifies the conjecture.

Rationale (proposer's reasoning):

Maslov dequantization gives $\max_i a_i = (1/\beta) \cdot \log \sum \exp(\beta a_i) + O(\log N / \beta)$, so any tropical identity that becomes exact at $\beta = \infty$ should pick up an $O(\log N / \beta)$ defect at finite β . The Example-2 witness ($\Delta=3.78$ at $\beta=5$, $n=8$) is consistent with $C \cdot \log(8)/\beta$ with C around 1.8, leaving generous headroom under $C=10$; if Δ instead decayed as $1/\sqrt{\beta}$ or stayed bounded, SC2's monotone-decrease prediction would be in trouble, so pinning the rate is the natural one-step inference toward SC2.

Taxonomy category: tropical_discrepancy_finite_beta (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7e81f73352cb4c31

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 seeds $\times$ 6 β -values (180 trials), **SUPPORTED** iff every trial satisfies $\Delta(f, \beta) \leq B(\beta) = 10 \cdot \log(8)/\beta$ (pass-rate = 1.0 at each β ; $\max \Delta/B(\beta) \leq 1.0$). **FALSIFIED** iff any single (seed, β) pair yields $\Delta > B(\beta)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.97	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.98	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.99	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -Maslov dequantization tropical Fourier transform min-plus convolution
-tropical polynomial self-convolution inverse temperature beta cyclic group Z_8 -min-plus
Fourier analysis doubling defect dequantization log-exp

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot row
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below the pivot
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]

def solve_linear_system(A, b):
    n = len(A)
    Ab = [row + [b[i]] for i, row in enumerate(A)]
    gaussian_elimination(Ab)

    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = (Ab[i][-1] - sum(Ab[i][j] * x[j] for j in range(i+1, n))) / Ab[i][i]
    return x
```

```

def log_sum_exp(h):
    max_val = max(h)
    return max_val + math.log(sum(math.exp(x - max_val) for x in h))

def maslov_dequantized_fourier_transform(f, beta):
    n = len(f)
    omega = math.exp(2 * math.pi / n)
    tft_f = [0] * n
    for k in range(n):
        if k == 0:
            sum_exp = log_sum_exp([-beta * f[j] for j in range(n)])
        else:
            sum_exp = log_sum_exp([-beta * (f[j] + omega**(j*k)) / 2 for j in range(n)])
        tft_f[k] = -(1/beta) * math.log(abs(sum_exp))
    return tft_f

def min_convolution(f):
    n = len(f)
    g = [0] * n
    for k in range(n):
        g[k] = min(f[i] + f[j] for i, j in enumerate(range(k+1, n)) if (i+j) % n == k)
    return g

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 8
    beta_values = [5, 10, 20, 50, 100, 200]
    results = []

    for beta in beta_values:
        f = [random.uniform(0, 1) for _ in range(n)]
        g = min_convolution(f)

        tft_f = maslov_dequantized_fourier_transform(f, beta)
        tft_g = maslov_dequantized_fourier_transform(g, beta)

        mfc_f = min(abs(tft_f[k]) for k in range(1, n))
        mfc_g = min(abs(tft_g[k]) for k in range(1, n))

        delta = abs(mfc_g - 2 * mfc_f)
        bound = 10 * math.log(n) / beta

        results.append({
            "beta": beta,
            "delta": delta,
            "bound": bound
        })

    max_delta = max(result["delta"] for result in results)
    pass_rate = sum(1 for result in results if result["delta"] <= result["bound"]) / len(results)

    return {
        "metric_name": "Delta",
        "metric_value": max_delta,
        "instances_tested": 6 * n,
        "n_max": n,
    }

```

```

        "conjecture_holds": pass_rate == 1.0,
        "counterexample": "" if pass_rate == 1.0 else f"beta={results[max(i for i, result in enumerate(results) if not result['conjecture_holds'])]}"
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(6, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_delta = sum(result["metric_value"] for result in results) / len(results)
    std_dev = math.sqrt(sum((result["metric_value"] - mean_delta)**2 for result in results) / len(results))
    pass_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_delta} std={std_dev} support_fraction={pass_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={result['counterexample']} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9f595dfe.py", line 109, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9f595dfe.py", line 75, in run_trial
    g = min_convolution(f)
         ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9f595dfe.py", line 64, in min_convolution
    g[k] = min(f[i] + f[j] for i, j in enumerate(range(k+1, n)) if (i+j) % n == k)
              ~~~~~^
ValueError: min() iterable argument is empty

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a ValueError in the min_convolution helper (empty min() iterable) before any trial data was generated, so the pre-registered support condition cannot be evaluated. | next: Fix the min-plus self-convolution implementation (iterate over all (i,j) with (i+j) mod 8 == k rather than the buggy enumerate(range(k+1,n))) and rerun the 30x6 trial grid.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	46849
2	preregistration	claude_max	opus	0	0	5855
3	novelty	claude_max	opus	0	0	3901
4	novelty	claude_max	opus	0	0	6248
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9741
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10359
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13335
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16778
9	judge	claude_max	opus	0	0	4564

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 117630 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/f26f8d71f2e7.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f26f8d71f2e7.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/f26f8d71f2e7.tar.gz` (if generated)